

ETEC PROFESSOR CAMARGO ARANHA

GROWLY

DIÁRIO DE BORDO / RELATÓRIO TÉCNICO SEMANAL

Semana 01 - 03 a 08 de junho de 2026

GROWLY 1.01 - primeiro PDC funcional

Equipe:

Thiago Nestor Afonso Dos Santos

Laís Silva Ferrari

Letícia Rodrigues Cardoso

Rebeca Carvalho Trindade

São Paulo - 2026

1. Identificação do registro

Período	03 a 08 de junho de 2026
Versão / etapa	GROWLY 1.01 - primeiro PDC funcional
Situação da semana	Desenvolvimento inicial do protótipo de identificação por imagem
Registro de versionamento	08/06/2026 - commit 52db5a2, "PDC Pronto"

2. Contexto e objetivo da semana

Esta semana marcou a transformação do conceito do Growly em um protótipo executável. Até então, o projeto já possuía base teórica, definição das PANCs como tema central, banco de dados estruturado e estudos sobre visão computacional, mas ainda faltava demonstrar de forma prática que uma imagem poderia ser recebida pelo sistema, preparada para a rede neural e classificada por um modelo treinado. A meta principal foi, portanto, construir um PDC simples, porém funcional, capaz de provar tecnicamente a viabilidade da identificação de plantas por inteligência artificial.

3. Planejamento técnico e escolha do ambiente

O desenvolvimento começou com a organização do ambiente de programação. Foram utilizados principalmente VS Code e PyCharm, com Python para os primeiros testes locais e JavaScript no navegador para a integração do modelo. A equipe precisou confirmar a versão do Python instalada, ajustar interpretadores e entender a diferença entre executar um modelo diretamente em Python e executá-lo dentro do navegador com TensorFlow.js. Essa etapa foi importante porque o objetivo final não era manter um script isolado de reconhecimento, mas aproximar a experiência de um aplicativo em que o usuário pudesse fornecer uma imagem e receber uma resposta imediatamente.

O primeiro caminho testado envolveu arquivos exportados pelo Teachable Machine, incluindo `keras_model.h5`, `labels.txt` e `metadata.json`. Em paralelo, foram utilizados os arquivos próprios para execução web - `model.json` e `weights.bin`. Também foi criado um script `scan.py` e mantidas imagens de teste separadas para verificar o comportamento do modelo fora da interface principal. Essa organização inicial deu origem a uma estrutura de protótipo com arquivos de treinamento, testes e uma versão web.

4. Primeiro modelo de visão computacional

Para validar o fluxo, foi treinado um modelo preliminar no Teachable Machine com classes de teste que ainda não representavam o catálogo final de PANCs: Planta01, Bambu, Espada-de-São-Jorge e uma classe de “não identificado/ND”. O objetivo não era obter precisão botânica, mas comprovar que o pipeline funcionava do começo ao fim: exportar o modelo, carregar os pesos, transformar a imagem para o tamanho esperado e exibir a classe de maior probabilidade.

O modelo utilizava imagens de entrada redimensionadas para 224 x 224 pixels. No JavaScript, a imagem era convertida em tensor, normalizada e expandida para incluir a dimensão de lote exigida pela rede. Em seguida, `model.predict()` retornava um vetor de probabilidades. O maior valor era localizado, associado ao rótulo correspondente e exibido como nome da classe e percentual de confiança. Esse procedimento estabeleceu a base lógica que permaneceria nas versões seguintes, mesmo quando a interface e o conjunto de classes fossem completamente modificados.

5. Construção da interface web do scanner

A primeira interface tinha como função principal permitir que o usuário escolhesse um arquivo de imagem. Foi criado um input de arquivo, uma área de resultado e a importação do TensorFlow.js por CDN. Aos poucos, o protótipo deixou de parecer apenas um teste técnico e começou a receber identidade visual do Growly. Foi adotada a paleta com verde escuro, verde médio, verde claro e bege, além de um card central que concentrava as ações do scanner.

Uma melhoria importante foi fazer a própria imagem escolhida aparecer dentro do card no lugar do texto de seleção. Isso exigiu trabalhar com `URL.createObjectURL/FileReader`, eventos de carregamento da imagem e regras de CSS para limitar dimensões sem deformar a fotografia. O ajuste de `object-fit` e do tamanho da área de preview foi necessário para impedir que imagens verticais ou horizontais fossem cortadas ou ultrapassassem o componente.

6. Inclusão da câmera do navegador

Como a proposta do Growly dependia da identificação no ambiente real, selecionar uma foto pronta não era suficiente. Foi então criada a função “Tirar foto”. O primeiro botão implementado ainda acionava o seletor de arquivo, pois utilizava a mesma lógica do upload. O comportamento foi corrigido com `navigator.mediaDevices.getUserMedia()`, acesso ao vídeo da webcam, elemento `<video>` e captura do quadro por meio de um `<canvas>`.

O fluxo passou a ser: solicitar permissão da câmera; mostrar a imagem ao vivo; permitir a captura; desenhar o frame atual no canvas; converter o resultado para imagem; substituir a visualização da câmera pela fotografia capturada; e liberar a etapa de análise. A correção foi considerada um marco porque o protótipo passou a representar de forma muito mais fiel a experiência planejada para o aplicativo final.

7. Problemas, depuração e aprendizados

Durante a semana ocorreram dificuldades com caminhos dos arquivos do modelo, versões de Python, abertura da câmera e apresentação das imagens. A equipe também precisou entender que o modelo exportado para TensorFlow.js possui arquivos dependentes entre si: `model.json` descreve a arquitetura e aponta para `weights.bin`, enquanto `metadata.json` registra informações do projeto e das classes. Se esses arquivos fossem movidos sem preservar os caminhos corretos, o carregamento falharia.

Outro aprendizado foi separar o problema de interface do problema de inteligência artificial. Muitas falhas iniciais não eram erros do treinamento, mas sim da forma como o navegador carregava o modelo, preparava a imagem ou manipulava a webcam. O processo de testar cada componente individualmente permitiu chegar a um protótipo funcional antes de substituir as classes temporárias por espécies reais.

8. Resultado da semana

Ao final, o GROWLY 1.01 possuía um scanner web funcional, capaz de receber imagem da galeria ou da câmera, exibir a fotografia selecionada/capturada e executar uma classificação por TensorFlow.js. O commit “PDC Pronto”, registrado em 08/06/2026, consolidou a primeira prova técnica do projeto. O protótipo ainda não estava pronto para

uso real, pois o modelo utilizava classes experimentais, mas demonstrou que a arquitetura proposta era viável e forneceu uma base concreta para a versão 1.02.

9. Síntese técnica e decisões registradas

A semana foi registrada como parte da evolução incremental do Growly. O objetivo do diário de bordo é preservar não apenas o resultado final, mas também as decisões, erros, testes e soluções que explicam por que a versão seguinte assumiu determinada forma. O estado final desta etapa pode ser resumido como: desenvolvimento inicial do protótipo de identificação por imagem.

O versionamento citado neste relatório é utilizado como evidência de continuidade e não substitui os testes práticos realizados durante o desenvolvimento. Quando não existe commit de uma nova versão no intervalo, o registro deixa essa ausência explícita para não atribuir ao período atividades que não estão documentadas.

10. Próximos passos definidos

Substituir as classes experimentais por PANCS reais, coletar um conjunto de imagens mais representativo, exportar um novo modelo do Teachable Machine e preparar a versão 1.02 para testes em celular e apresentação.

11. Observação metodológica do diário de bordo

Este relatório foi reconstruído a partir do histórico técnico do projeto, das versões preservadas, dos registros de testes e do histórico de versionamento. Nas semanas em que há evidências precisas, são descritos os procedimentos, problemas e soluções em maior detalhe. Nos períodos sem registro formal de nova versão, o texto indica explicitamente a ausência de mudança documentada, evitando preencher lacunas com acontecimentos não comprovados.

A descrição técnica utiliza os nomes reais de arquivos, tecnologias e componentes adotados no Growly para que o diário também funcione como memória de desenvolvimento e apoio à documentação final do TCC.